Use if, elif, else statements to make decisions

Now that we know about variables, expressions, functions, data types, comparators, and logical operators, we can perform exciting actions in our scripts based on their values using branching. Which is exactly what we'll learn about in this video! Branching describes the ability of a program to alter its execution sequence. This is a key component to writing useful scripts. Branching uses IF statements based on certain conditions. IF is a reserved keyword that sets up a condition in Python. IF statements, also known as conditional statements, are just like using the word "if" in everyday life. For example, if it's before noon, you'll greet someone by saying good morning rather than good afternoon or good evening. If it's raining outside, you might choose to carry an umbrella. And if it's snowing, you'll probably wear a jacket. Here's an example of this concept in a business context. At a company, new employees can choose their usernames. However, the usernames need to fit a given set of guidelines. Maybe a valid username requires at least eight characters.

As the data professional at this business, you're tasked with writing a program that will tell the user if their choices are valid or not. To accomplish this task, we'll write a function. The goal is to define the function so that it generates a username hint using an IF statement. As a reminder, the built-in len() function will return the length of an object, and that can be paired with the less-than comparator to identify usernames that don't

meet the criteria. Great, now your function checks whether the length of the username is less than eight. If it is, the function prints a message saying that the username is invalid. Let's review our IF statement.

We write the keyword IF followed by the condition that we want to check for, followed by a colon. After that, we have the body of the IF block, which is indented further to the right. Here's a very important point - The body of the IF block will only execute when the condition evaluates to True; otherwise, it does not execute. What this means is - if you run an IF block and the argument conditions are not met, the indented code beneath it gets ignored. The IF statement is a useful construct in Python syntax. But what if we could extend it to make it even more powerful? What if we want the computer to do something... ELSE?

ELSE is a reserved keyword that executes when preceding conditions evaluate as False. The ELSE statement lets us set a piece of code to run only when the condition of the IF statement is False. Here's an everyday example: IF you're hungry, you eat. But if you're not hungry, if that concept is FALSE, then you'll do something ELSE. Maybe you'll make another choice and take a nap. Think about our company username example. Maybe now we want to print a message when a username is valid.

The function can now go in different directions depending on the length of the username. If the username is not long enough, a message indicates that it's invalid. But if the function verifies that the username is long enough, it will print a message saying that it's valid, which is dictated by the ELSE statement. Notice the structure of the function right now. The IF statement is indented in the body of the function, and the action we want to happen if that statement is True is indented beneath it. We could write

as many lines as we want here, and as long as they're all indented beneath the IF statement, they'll all execute when that IF statement is True. Then we have the ELSE statement.

Note that it's unindented to the same level as the IF statement. An IF statement and its corresponding ELSE statement are always written at the same level. Beneath the ELSE statement, we indent once more to indicate that this is what must execute when the IF statement is NOT True. Sometimes, you don't need to add an ELSE statement because that logic is already built into the code. Let's explore a new operator that will help with this. We're going to use a new operator, the modulo. Represented by the percentage sign, the Modulo is an operator that returns the remainder when one number is divided by another.

The division between integers yields two results which are both integers: the quotient and the remainder. So, for an integer division between five and two, the quotient is two, and the remainder is one. For an integer division between eleven and three, the quotient is three, and the remainder is two. Even numbers are all multiples of two, which means the remainder of the integer division between an even number and two is always going to be zero. So, the modular division of ten by two is zero. Let's review an example. This function checks whether a number is even by dividing it by two and checking that the remainder is zero, using the modulo operator.

If the remainder is zero, the function will return True. Now here's the interesting part. You can put an ELSE statement here. That would work. But it's not strictly necessary, because of the way IF statements work. Remember, when the IF statement evaluates to

True, the code indented beneath it will execute. But when the IF statement evaluates to False, nothing indented beneath it will execute.

The code will then continue running until it gets to the end of the function. Let's try entering the odd number 19 using the is-even function we defined. The function returned False, because 19 modulo two does not evaluate to True, so the code indented beneath the IF statement doesn't execute, and then the function continues running. In this case, the only remaining code in the function is "return False." So that's what it does. It returns False. At first, you might prefer to include the ELSE statement in such instances, and that's okay.

It's important to know that both ways are correct. But, keep in mind this technique can only be used when returning a value inside the IF statement. To recap, an IF statement branches the execution based on a specific condition being True, and the ELSE statement sets a piece of code to run only when the condition of the IF statement is False. For situations where there are more conditions to consider, the ELIF statement, short for ELSE-IF, is useful. The ELIF keyword is a reserved keyword that executes subsequent conditions when the previous conditions are not True. It's Python's way of saying "if the previous conditions were not True, then try this condition." Let's consider an example to better understand ELIF.

It is likely the case that the weather might influence what you choose to do with your afternoon. If the weather is nice, you might go to the park. If it's raining, you might go to the movies instead. Depending on which of these activities you choose to do, you also need to decide how to get there. And, the activity may determine your mode of

transportation! So, the choices you make depend on different conditions at each point. These are IF, ELSE-IF statements you may encounter in daily life.

Let's return to the username validation example. Perhaps now we want to limit how long the username can be. Maybe our business has a rule that usernames longer than 15 characters aren't allowed. Let's type our first condition: usernames of less than eight characters are invalid. Now we want to add another condition to this, limiting the username to a maximum of 15 characters. Notice there are two ELSE statements. The first ELSE is the second course of action if the first condition is not met and the length of the username is greater than or equal to eight.

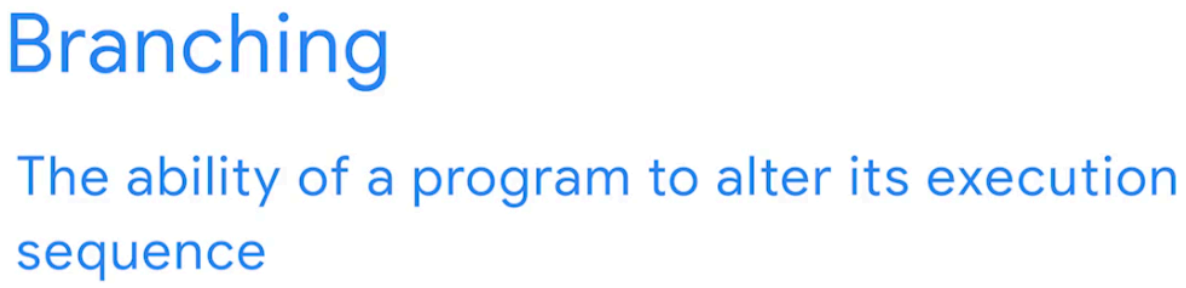
In other words, if the first IF statement is False, then the code executes the first ELSE statement. This ELSE statement has two more nested conditions: IF and an ELSE. The indentations make the relationships between the different branching statements easier to read, but the nesting adds some complexity. Remember, you can choose to use as many or as few spaces as you want for the indentation, but generally it's best to use four spaces for readability. And, it's important to be consistent. To avoid unnecessary nesting and to make the code clearer, Python's ELIF keyword lets us handle more than two comparison cases. In fact, the ELIF keyword allows us to handle an unlimited number of comparison cases!

The ELIF statement is similar to the IF statement. The abbreviation for ELSE-IF prevents a lot of nested IF and ELSE statements. If all the above conditions are False, then the final ELSE statement executes. Now let's run our function on a really long username. This script works just the same as the one with nested IF-ELSE comparison

I just demonstrated, but is much easier to understand. Let's examine it. The function first checks whether the username is less than eight characters long.

If that's the case, it prints a message. Next, if the username has at least eight characters, the function then checks if it's longer than 15 characters and prints a message if that's True. If neither of the above conditions are met, the function prints a message indicating that the username is valid. Now you know how to use IF, ELIF, and ELSE statements inside functions! This kind of branching is super-helpful when determining the flow of your scripts. Use branching to pick between different pieces of code to execute, making your script pretty flexible and efficient. Branching also helps with all kinds of practical things, such as bin data based on its value, backup files, or to only allow login access to a server during certain times of day.

Any time your program needs to make a decision, you can specify its behavior with a branching statement. Now you have a strong foundation to build branches in code, which is going to enable you to do a lot of useful work in Python as a data professional. You've learned so much already, and I hope you're enjoying the discovery as much as I am!



`if`

A reserved keyword that sets up a condition in Python

`else`

A reserved keyword that executes when preceding conditions evaluate as False

Modulo

An operator that returns the remainder when one number is divided by another

Question

Fill in the blank: _____ describes the ability of a program to alter its execution sequence.

- ○ Commenting
- ◉ Branching
- ○ Refactoring
- ○ Comparing

Correct
Branching describes the ability of a program to alter its execution sequence.

Continue Skip

`elif`

A reserved keyword that executes subsequent conditions when the previous conditions are not true

The `elif` keyword lets us handle more than two comparison cases.

Uses of branching

- Bin data based on its value
- Backup files
- Restrict login access